FIAP | ENGENHARIA DE SOFTWARE | 2ESPY

TOTVS AdvisorMind

Sprint 4 Java - FINAL

Integração ao modelo relacional de Database Design

Integrante (documentação original)	RM
Enzo Fernandes Ramos	563705
Felipe Henrique de Souza Cerazi	562746
Gustavo Peaguda de Castro	562923
Lorenzo Coque	563385

Professor: Rafael Desiderio. São Paulo, 2026.

Nome oficial da equipe: não comprovado nos materiais; confirmar com os integrantes. Solução: TOTVS AdvisorMind.

Versão técnica 3.0.0 | Java 21 | Spring Boot | Maven | JDBC

Implementação e testes locais concluídos. Validação em Oracle remoto, chamada real à OpenAI e integração visual de Front-End continuam pendentes.

Evidências: 34 testes locais aprovados; 2 testes Oracle opcionais não executados; 52 verificações HTTP com três inicializações.

Sumário

1. Objetivo, escopo e fontes 3

2. Diagnóstico e decisões de evolução 4

3. Arquitetura, transações e segurança de dados 5

4. Modelo relacional - seis tabelas 6

5. Modelo Java e regras preservadas 7

6. Inventário de classes e camadas 8

7. Estratégia dos repositórios e IA 9

8. Contrato REST - cadastros e repositórios 10

9. Contrato REST - reuniões e análises 11

10. Corpos, validações e exemplos de execução 12

11. Evidências JSON e resultados persistidos 13

12. Revisão do SQL e propostas separadas 14

13. Executar localmente e configurar Oracle 15

14. Testes executados e limites da validação 16

15. Oracle, Front-End e checklist acadêmico 17

16. Pacote final e rastreabilidade 18

1. Objetivo, escopo e fontes

O AdvisorMind apoia a revisão de reuniões comerciais a partir de transcrições. SmartBriefing organiza um resumo; Conversational Analytics interpreta a dinâmica; VoC Tracker destaca dores, sugestões e oportunidades. Os resultados apoiam o profissional, sem substituir sua decisão. Não há reconhecimento de áudio implementado.

Fontes e prioridade

Fonte	Uso
SQL enviado no chat em 19/10/2026	Referência prioritária das seis tabelas, colunas, PKs, FKs, unicidades, tipos e limites.
Projeto Sprint4 anterior (2.0.0)	Base Maven/Java21/SpringBoot/JDBC, regras úteis, prompts de IA, menu e estrutura de pacotes.
PDF anterior e documentação Sprint3	Integrantes/RMs, objetivo e evolução da arquitetura. PDF atualizado para o código final.
Regulamento Sprint3/4 TOTVS	Java Sprint4 nas páginas físicas28-29 (rodapé20): documentação20, Model10, DAO/Service20, REST30 e boas práticas20.
prompt.txt	Instruções de integração, preservação de fontes, validação e entrega final.

O SQL foi colado no chat, não anexado como arquivo. sql/recebido.sql transcreve os comandos e literais na sequência enviada, com espaços/formatação normalizados. Propostas auxiliares estão em arquivos separados. O pacote anterior não foi sobrescrito, e nenhum banco remoto foi alterado.

Funcionalidades finais

CRUD de clientes, vendedores e dois tipos de repositório; CRUD relacional de reuniões; indicadores acadêmicos e estatísticas; geração/persistência de três tipos de análise de IA; prévia completa não persistida; consulta/exclusão de resultados; console adaptado; validação e tratamento de erros. O Front-End não foi fornecido e sua integração não é alegada.

2. Diagnóstico e decisões de evolução

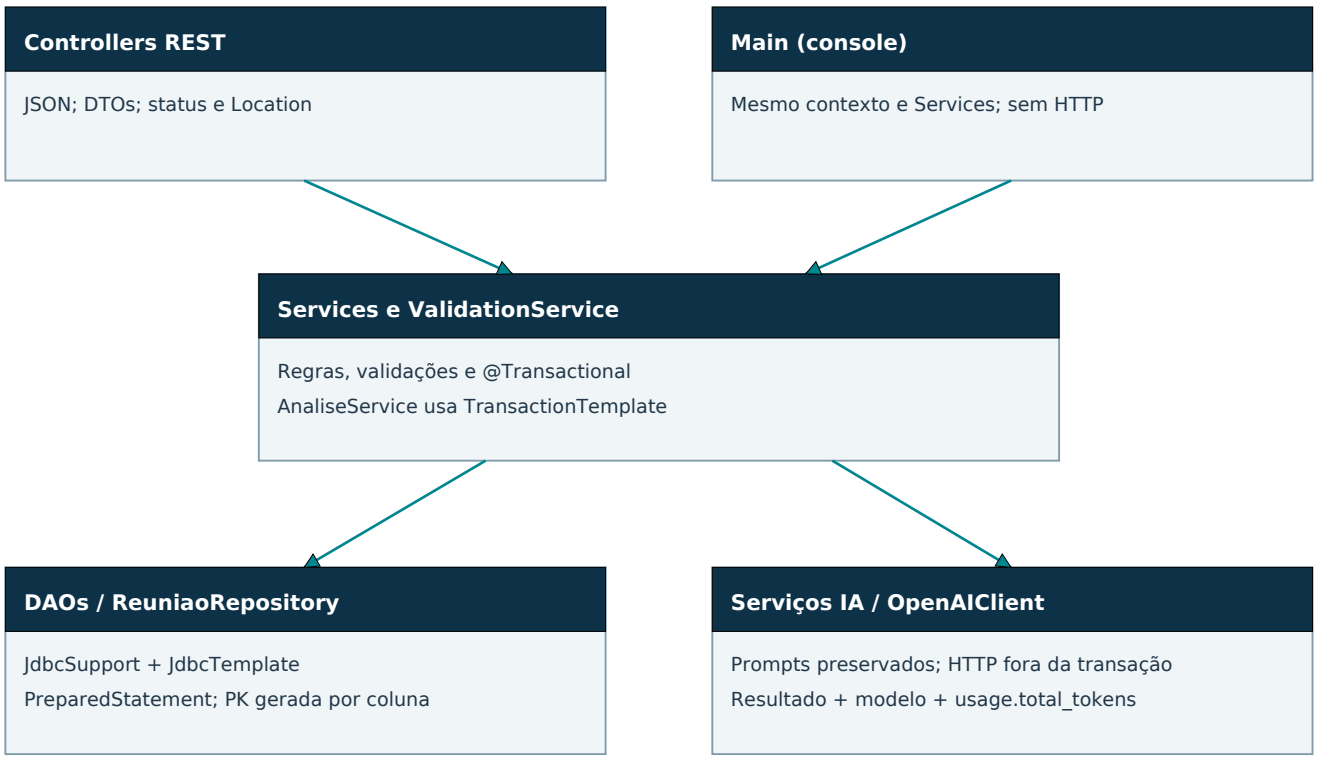
Divergência anterior	Decisão final
T_TOTVS_VENDEDOR com ID manual e ticket	Usar VENDEDOR com identidade e FK; ticket não é coluna nem campo do cadastro.
Cliente só com nome/empresa	Adicionar ID e dataNascimento; persistência JDBC; UNIQUE(nome,empresa).
Reunião em arquivo JSON com objetos embutidos	Migrar para REUNIAO; três FKs, data/status e ID gerado.
Transcrição de até60000	Limite4000 caracteres e bytes UTF-8; rejeição sem cortar.
Análise retornada sem gravação	ANALISE_IA com tipo, texto, modelo, data/hora, tokens e FK.
RESULTADO=250 versus prompts extensos	Compacto persistido; prévia completa sem gravação. Exceder limite falha explicitamente.
Contadores/JSON sem semântica operacional definida	Contadores dos repositórios derivados das FKs; JSON CLOB só índice de IDs.
Comissão dependia de ticket persistido	Overload BigDecimal para simulação explícita; sem ticket não há valor calculado.
Testes do modelo híbrido	Substituídos por testes relacionais, rollback, IA persistida e probes Oracle opt-in.

Reaproveitamento

Mantidos o artefato Maven, Java21, SpringBoot3.5.7, driver Oracle original, pacotes e três entidades centrais. Preservadas as regras de perfil, senioridade, comissão (com entrada explícita), incremento de reuniões, temperatura e duração. Prompts completos permanecem na prévia. Menu e formatter foram adaptados; relatório/estatísticas agora leem referências relacionais.

ReuniaoRepository foi evoluído para JDBC. JsonUtil deixou de ler/gravar arquivos e agora gera apenas o índice derivado. Os executáveis manuais antigos que dependiam do contrato obsoleto foram substituídos por JUnit; originais permanecem na entrega anterior. Não há migração automática de JSON sem datas e FKs conhecidas.

3. Arquitetura, transações e segurança de dados



Pacotes: entities, dto, controller, service, dao, repository, conexoes, exception e util. As respostas expõem somente campos efetivos do modelo; a prévia usa DTO próprio. ApiErrorHandler centraliza erros; SQL não fica nos controllers. Spring gerencia DataSource, conexões e transações; JdbcTemplate fecha recursos JDBC e participa da conexão vinculada à transação.

Operações atômicas

Cadastro/alteração/exclusão de vendedor atualiza seu agrupamento na mesma transação. Reuniões atualizam linha relacional, CLOB e contador juntos. SELECT FOR UPDATE bloqueia os registros envolvidos; pais múltiplos são bloqueados por ID ordenado. Falha intermediária reverte as alterações. Duas provas automatizadas forçam falha após INSERT e confirmam ausência de gravação parcial.

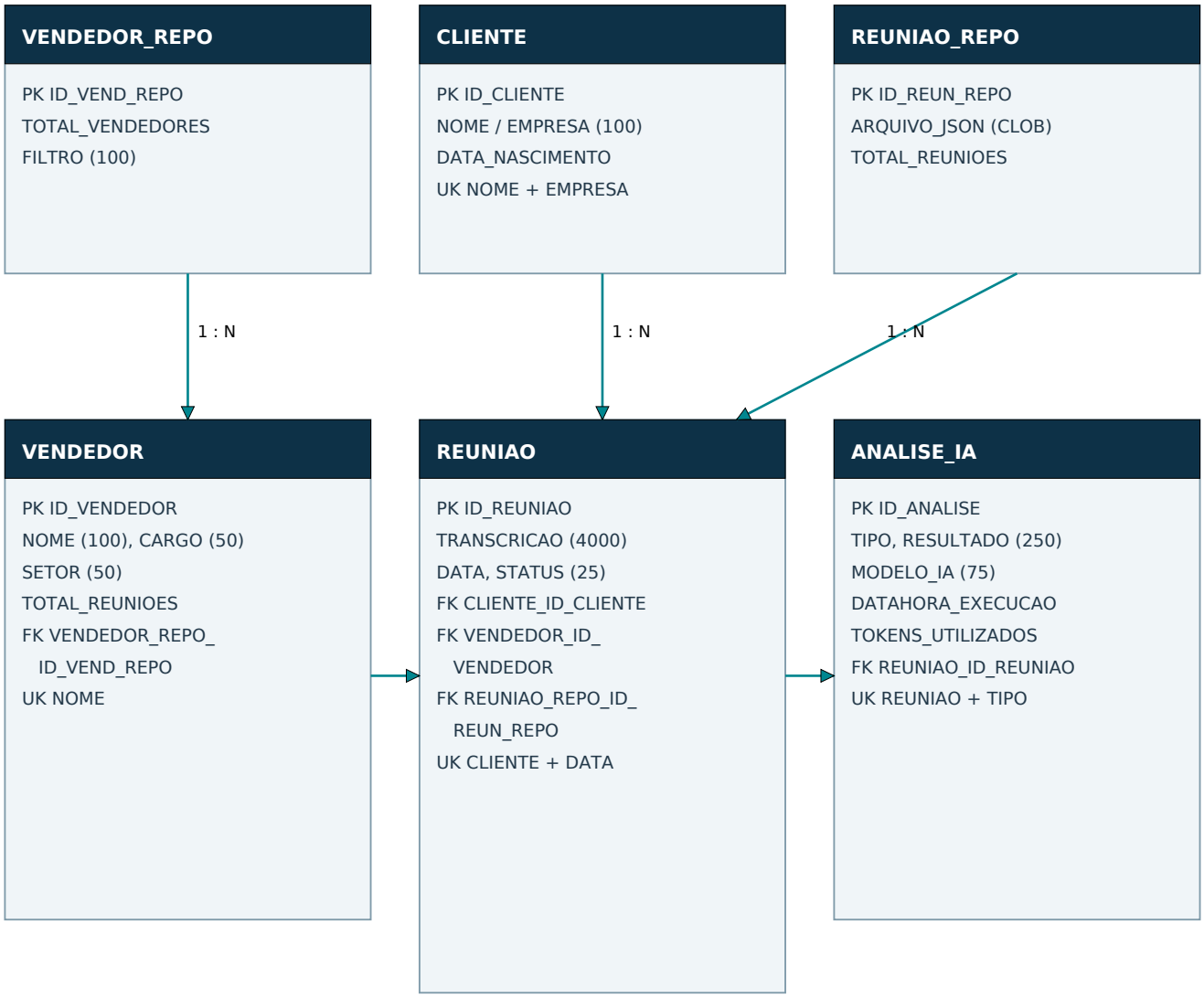
Análise externa e gravação

O HTTP ocorre fora da transação. Depois da resposta, uma transação curta bloqueia a reunião, compara conteúdo/cliente/vendedor/data, verifica duplicidade e insere. Mudança concorrente gera409; duas chamadas concorrentes podem consumir custo externo, mas o banco permite apenas uma linha por reunião/tipo. Rollback não desfaz cobrança do provedor. Não há retry automático.

Integridade e configuração

FKs impedem excluir pais com dependentes (409), sem cascata oculta. Configuração de senha/chave é externa. Sem SQL/stacktrace no erroHTTP. Listener padrão em loopback; autenticação, autorização e CORS para implantação pública não foram definidos no escopo acadêmico.

4. Modelo relacional - seis tabelas



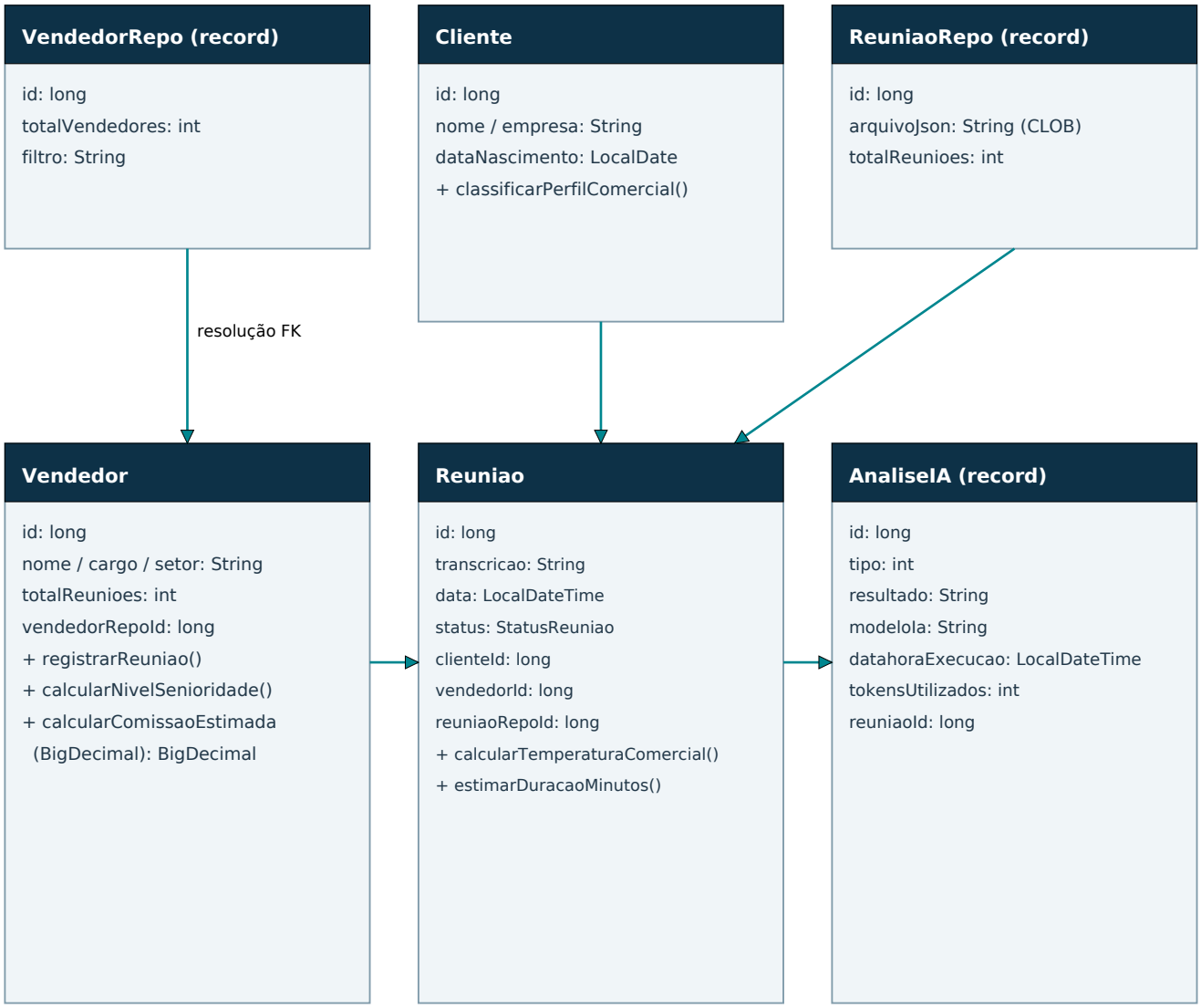
Setas seguem pai → dependente; cada filho exige exatamente um pai por FK e cada pai pode ter zero ou muitos filhos. PKs são identidades geradas pelo banco. Não há FK para arquivos JSON e não existe T_TOTVS_VENDEDOR nesta versão. As associações do Java usam os IDs reais, não snapshots.

Datas, limites e identidade

Oracle DATE guarda até segundos e não guarda fuso. Cliente usa LocalDate. Reunião usa LocalDateTime sem offset e sem frações. Execução IA usa convenção UTC sem offset. UNIQUE(cliente,data) compara a data/hora inteira, permitindo reuniões em segundos diferentes do mesmo dia. API aceita anos1..9999 e IDs long positivos.

Para VARCHAR2, a API valida tanto caracteres quanto bytes UTF-8, conservadoramente para AL32UTF8/BYTE. H2 valida comprimentos como caracteres; NLS, charset e collation reais do Oracle ainda precisam de teste. IDs são recuperados pela coluna solicitada ao PreparedStatement, evitando receber ROWID no lugar da identidade.

5. Modelo Java e regras preservadas



Setas indicam resolução relacional de IDs, não objetos embutidos ou herança. Cliente, Vendedor e Reuniao mantêm construtores vazio/completo, atributos privados, getters/setters e métodos de regra. Novos modelos de repositório/análise são records imutáveis.

Heurísticas, sem alegar política oficial

Perfil comercial por termos do nome da empresa; senioridade por total histórico informado; comissão baseada em cargo/senioridade/ticket explícito; temperatura por palavras-chave; duração por130 palavras/minuto. São regras acadêmicas herdadas e não métricas oficiais TOTVS. A identificação de ME utiliza palavra inteira.

Sem TICKET_MEDIO no DDL, a comissão não pode vir de dado persistido inexistente: GET indicadores aceita ticketMedio opcional para simular com BigDecimal/HALF_EVEN. Sem ticket retorna null e motivo. O método no-arg legado lança erro explícito, em vez de inventar zero. registrarReuniao é preservado e exposto como ação separada; cadastrar reunião não altera automaticamente o histórico informado do vendedor.

6. Inventário de classes e camadas

Pacote / componente	Função e dependências
AdvisorMindApplication / Main	Inicialização HTTP/console; console usa os Services. ConsoleFormatter mantém formatação.
entities	Cliente, Vendedor, Reuniao, VendedorRepo, ReuniaoRepo, AnaliseIA; enums StatusReuniao e TipoAnalise.
dto	ClienteRequest, VendedorRequest, ReuniaoRequest, VendedorRepoRequest, ReuniaoRepoRequest; OpenAICompletion e AnalisePreview.
controller	ClienteController, VendedorController, ReuniaoController, VendedorRepoController, ReuniaoRepoController e AnaliseController. Sem SQL.
Services CRUD	ClienteService, VendedorService, ReuniaoService, VendedorRepoService e ReuniaoRepoService: validação, referências, transações e sincronização.
IA / relatórios	AnaliseService coordena HTTP e persistência; SmartBriefingService, ConversationalAnalyticsService, VoCTrackerService mantêm prompts. OpenAIClient valida envelope/metadados. RelatorioService calcula estatísticas.
ValidationService	Bean Validation mais limites em bytes/IDs/existência; mensagens sem conteúdo sensível.
dao / repository	JdbcSupport gerencia operações parametrizadas via JdbcTemplate; ClienteDao, VendedorDao, VendedorRepoDao, ReuniaoRepoDao, AnaliseIADao e ReuniaoRepository fazem mapeamentos explícitos.
conexoes / util / exception	ConexaoFactory legada via ambiente; API usa DataSource gerenciado. JsonUtil só gera índice. ApiException e ApiErrorHandler centralizam erros.

Atualização de conteúdo já analisado

Reunião com análise persistida bloqueia mudanças em transcrição, cliente, vendedor e data. Status ou agrupamento podem mudar. Exclusão de análise libera nova execução daquele tipo; não existe PUT para reescrever texto/modelo/tokens manualmente. Isso preserva a proveniência sem criar dados artificiais.

7. Estratégia dos repositórios e IA

Uma fonte de verdade

REUNIAO é canônica. REUNIAO_REPO.ARQUIVO_JSON contém somente {"reunioes":[IDs]}, em CLOB, e TOTAL_REUNIOES é sua contagem. Não há arquivo JSON local ativo. POST de repositório inicia vazio; PUT {} recalcula a projeção. GET não repara silenciosamente dados carregados por SQL externo.

VENDEDOR_REPO agrupa vendedores pela FK e mantém TOTAL_VENDEDORES derivado. FILTRO é descrição livre, não SQL executável. O contador de VENDEDOR é diferente: representa o histórico declarado e pode incluir reuniões não cadastradas na tabela atual.

Tipos e dois modos de análise

TIPO inteiro	Rota	Foco
1	smart-briefing	Resumo executivo, decisões e próximos passos.
2	conversational-analytics	Sentimento, dinâmica, temas e proporção de fala.
3	voc-tracker	Dores, sugestões e oportunidades.

Persistido: solicita parágrafo compacto, valida limite250 caracteres/bytes e salva exatamente o conteúdo recebido. modeloIa e usage.total_tokens vêm do provedor; ausência ou valores inválidos impedem gravação. datahoraExecucao é gerada pelo sistema em UTC com segundos. Resultado excedente retorna502 sem truncar.

Prévia: preserva os prompts extensos da Sprint3 e devolve resposta completa com persistida:false; não salva. Nenhum resultado completo é alegado como salvo no campo250. Ambas as operações enviam transcrição ao provedor e podem ter custo. A recomendação formal é ampliar RESULTADO para CLOB se a equipe decidir persistir análises completas; este DDL não foi alterado.

Falhas externas

Sem chave503; timeout504; limite/indisponibilidade503; resposta inválida, metadados ausentes, resposta incompleta ou erro externo502. Modelo precisa caber em75 caracteres/bytes. Tokens precisam ser inteiros não negativos informados pelo provedor. Timeout conexão10s/requisição45s. Não há resposta simulada em produção e não há chamada paga nos testes automatizados.

8. Contrato REST - cadastros e repositórios

Base local: http://127.0.0.1:8080. JSON UTF-8. IDs gerados, nunca recebidos no corpo. Campos desconhecidos, coerção de escalares e frações para inteiros são rejeitados. Status gerais:400 inválido,404 ausente,409 unicidade/FK/dependência,500 falha inesperada,405 método e415 mídia incompatível.

Verbo / URI	Entrada / finalidade	Sucesso / erros
POST /api/clientes	ClienteRequest; Criar com identidade gerada	201 / 400, 404, 409, 500
GET /api/clientes	Nenhum; Listar	200 / 500
GET /api/clientes/{id}	id positivo; Consultar	200 / 400, 404, 500
PUT /api/clientes/{id}	id + ClienteRequest; Atualizar	200 / 400, 404, 409, 500
DELETE /api/clientes/{id}	id positivo; Excluir; dependências impedem exclusão	204 / 400, 404, 409, 500
POST /api/vendedores	VendedorRequest; Criar com identidade gerada	201 / 400, 404, 409, 500
GET /api/vendedores	Nenhum; Listar	200 / 500
GET /api/vendedores/{id}	id positivo; Consultar	200 / 400, 404, 500
PUT /api/vendedores/{id}	id + VendedorRequest; Atualizar	200 / 400, 404, 409, 500
DELETE /api/vendedores/{id}	id positivo; Excluir; dependências impedem exclusão	204 / 400, 404, 409, 500
POST /api/vendedor-repos	VendedorRepoRequest; Criar com identidade gerada	201 / 400, 404, 409, 500
GET /api/vendedor-repos	Nenhum; Listar	200 / 500
GET /api/vendedor-repos/{id}	id positivo; Consultar	200 / 400, 404, 500
PUT /api/vendedor-repos/{id}	id + VendedorRepoRequest; Atualizar	200 / 400, 404, 409, 500
DELETE /api/vendedor-repos/{id}	id positivo; Excluir; dependências impedem exclusão	204 / 400, 404, 409, 500
POST /api/reuniao-repos	ReuniaoRepoRequest; Criar com identidade gerada	201 / 400, 404, 409, 500
GET /api/reuniao-repos	Nenhum; Listar	200 / 500
GET /api/reuniao-repos/{id}	id positivo; Consultar	200 / 400, 404, 500
PUT /api/reuniao-repos/{id}	id + ReuniaoRepoRequest; Regerar índice e contador	200 / 400, 404, 409, 500
DELETE /api/reuniao-repos/{id}	id positivo; Excluir; dependências impedem exclusão	204 / 400, 404, 409, 500

DELETE204 não tem corpo; GET de lista vazia retorna[]. POST retorna201 com Location apontando ao ID gerado. PUT substitui campos editáveis. POST/PUT de ReuniaoRepo usa {}, pois índice/contador não são arbitrários. Modelo completo dos corpos e respostas está em docs/API.md.

9. Contrato REST - reuniões e análises

Verbo / URI	Entrada / finalidade	Sucesso / erros
POST /api/reunioes	ReuniaoRequest; Criar com identidade gerada	201 / 400, 404, 409, 500
GET /api/reunioes	Nenhum; Listar	200 / 500
GET /api/reunioes/{id}	id positivo; Consultar	200 / 400, 404, 500
PUT /api/reunioes/{id}	id + ReuniaoRequest; Atualizar	200 / 400, 404, 409, 500
DELETE /api/reunioes/{id}	id positivo; Excluir; dependências impedem exclusão	204 / 400, 404, 409, 500
GET /api/vendedores/{id}/indicadores	id; query ticketMedio opcional; Senioridade; simulação opcional de comissão	200 / 400, 404, 500
POST /api/vendedores/{id}/registrar-reuniao	id; sem corpo; Incrementar contador histórico explícito	200 / 400, 404, 409, 500
GET /api/reunioes/estatisticas	Nenhum; Contagens de reuniões/empresas e maior transcrição	200 / 500
GET /api/reunioes/{id}/indicadores	id positivo; Temperatura, duração e perfil acadêmicos	200 / 400, 404, 500
POST /api/reunioes/{id}/analises/{tipo}	id; tipo; sem corpo; Gerar análise compacta e persistir	201 / 400, 404, 409, 500, 502, 503, 504
POST /api/reunioes/{id}/analises/{tipo}/previa	id; tipo; sem corpo; Gerar análise completa sem persistir	200 / 400, 404, 500, 502, 503, 504
GET /api/analises	Nenhum; Listar resultados persistidos	200 / 500
GET /api/analises/{id}	id positivo; Consultar resultado persistido	200 / 400, 404, 500
GET /api/reunioes/{id}/analises	id positivo; Listar resultados da reunião	200 / 400, 404, 500
DELETE /api/analises/{id}	id positivo; Excluir para permitir nova análise/exclusão da reunião	204 / 400, 404, 500

Tipo na rota: smart-briefing, conversational-analytics ou voc-tracker. Outro valor retorna404. Não há corpo de análise: transcrição vem da reunião persistida. Resultado persistido retorna201; prévia retorna200. Não há POST de metadados de análise enviados manualmente nem PUT que altere proveniência.

GET indicadores de vendedor aceita query ticketMedio opcional (0..999999999999.99, até duas casas). Sem parâmetro a comissão é null com explicação; com parâmetro retorna simulação. As demais rotas não usam parâmetros de consulta. IDs são long positivos; registros referenciados devem existir.

10. Corpos, validações e exemplos de execução

DTO / campos	Validações
ClienteRequest: nome, empresa, dataNascimento	Texto não branco até100 caracteres/bytes; nascimento não futuro e ano1..9999, formato yyyy-MM-dd.
VendedorRepoRequest: filtro	Opcional até100 caracteres/bytes; contador controlado pelo servidor.
ReuniaoRepoRequest: {}	Sem campos editáveis; criação vazia ou atualização da projeção.
VendedorRequest: nome, cargo, setor, totalReunioes, vendedorRepoId	Nome até100, cargo/setor50; textos obrigatórios; total inteiro0..2147483647; repo obrigatório existente.
ReuniaoRequest: transcricao, data, status, clienteId, vendedorId, reuniaoRepoId	Texto não branco até4000 caracteres/bytes; todos obrigatórios; ano1..9999, data ISO sem fuso/fração; status AGENDADA/ATENDIDA/CANCELADA.

Ordem para criar dados

Crie vendedor-repos e reuniao-repos, depois cliente e vendedor usando o ID do agrupamento; por último reunião com as três FKs reais. A coleção Postman registra os IDs automaticamente. requests.http tem variáveis para preencher. Nenhum exemplo exige o ID manual do modelo anterior.

Reunião - requisição real no smoke

```
{
  "data": "2026-09-01T10:00:00",
  "status": "ATENDIDA",
  "clienteId": 1,
  "vendedorId": 1,
  "reuniaoRepoId": 1
}
```

Campo transcricao da mesma requisição: Cliente: "Gostei", vamos avancar.
Vendedor: proposta! ação □

As respostas de reunião contêm os campos acima mais id gerado; clienteId/vendedorId/reuniaoRepoId são referências reais. A ausência de dados obrigatórios é erro, sem defaults inventados.

11. Evidências JSON e resultados persistidos

Resposta real de criação da reunião


```
{
  "id": 1,
  "transcricao": "[valor completo em http-smoke.json]",
  "data": "2026-09-01T10:00:00",
  "status": "AGENDADA",
  "clienteId": 1,
  "vendedorId": 1,
  "reuniaoRepoId": 1
}
```


201 + Location da reunião gerada. Somente o valor longo da transcrição foi abreviado acima para leitura; os dados completos estão em docs/evidencias/http-smoke.json, sem alteração da persistência.

Análise persistida no teste controlado


```
{
  "id": 1,
  "tipo": 1,
  "resultado": "FIXTURE DE TESTE: cliente interessado em demonstracao.",
  "modeloIa": "controlled-local-fixture",
  "datahoraExecucao": "2026-10-20T01:41:56",
  "tokensUtilizados": 77,
  "reuniaoId": 1
}
```


É uma fixture explicitamente produzida pelo provedor HTTP local de teste, não pela OpenAI real. Tokens77 e modelo controlled-local-fixture vieram desse envelope e comprovaram a gravação dos metadados. Nenhum token do provedor real foi inventado ou alegado como testado.

Erro real, sem chave


```
{
  "timestamp": "2026-10-20T01:41:52.753407650Z",
  "status": 503,
  "mensagem": "Integração de IA não configurada."
}
```


As evidências também incluem conflitos409, referência ausente404, texto excessivo400, exclusões204 e recuperação das seis tabelas após reiniciar. Esses registros HTTP não são prints de telas do Front-End.

12. Revisão do SQL e propostas separadas

Achado	Impacto / tratamento
Limpeza anunciada e ausente	Nenhum DROP foi acrescentado. Roteiro recebido só cria/popula/consulta.
CREATE e carga não idempotentes	Rodar original uma vez em schema vazio; reexecução em schema ocupado pode falhar ou duplicar repositórios se o cliente continuar após erros.
FKs1..5 pressupõem identidade inicial	API usa IDs recuperados. Auxiliar DEMO procura chaves naturais e usa RETURNING.
Contadores de vendedor-repo	Recebidos5,2,3,1,4; relações reais2,1,1,1,0. Sincronização proposta separada.
Filtros e associações não coincidem	Gabriel/Serviços ligado a repo descrito como Tecnologia. FILTRO tratado como metadado, sem inventar reassociação.
JSONs e totais de reunião-repo	Cada repo tem uma reunião; totais/nomes no JSON não representam as FKs. Aplicação usa projeção derivada de IDs.
DATE e unicidade	Comparação por cliente/data-hora completa. Não reduzir para calendário diário.
RESULTADO250	Compacto validado e prévia completa; recomendação formal CLOB se necessário.
Gemini/tokens da carga	Exemplos acadêmicos, sem prova de chamadas externas; não usados como evidência de execução.
Cinco categorias DQL	Preservadas: cálculo, INNER JOIN, JOIN com GROUP BY, LEFT JOIN com WHERE/ORDER BY e HAVING.
Repetições ao final / comentários	Relatórios repetidos não alteram dados. Comentários de tabelas existem; faltam comentários de colunas da rubrica Database/Sprint3.

sql/recebido.sql mantém os comandos recebidos; oracle-create.sql separa apenas DDL; dql-academico.sql separa as cinco consultas. proposta-demo-oracle.sql é suplementar, não substitui a carga oficial de cinco linhas por tabela. proposta-sincronizar-repos-oracle.sql corrige somente projeções, manualmente e sem concorrência (Oracle12.2+ para JSON_ARRAYAGG).

Na validação local, o recebido foi executado em H2/Oracle mode com a adaptação de identidade. Terminou com cinco registros por tabela. O segundo CREATE foi rejeitado, comprovando o problema de reexecução. Oracle real continua pendente.

13. Executar localmente e configurar Oracle

Local

JDK21 e Maven3.6.3+; valide java -version e mvn -version. Abra a pasta do pom.xml no IntelliJ. Primeiro build precisa de rede para baixar dependências; não use -o antes de preparar o cache.

```
mvn clean verify
java -jar target/TOTVS-AdvisorMind-3.0.0.jar
```

API em127.0.0.1:8080. H2 local em data/advisormind-final.mv.db; schema sem dados artificiais automáticos. Nome separado da versão2.0.0 evita reutilização silenciosa do esquema antigo. Nenhum JSON legado é importado automaticamente. Main abre console; AdvisorMindApplication abre HTTP.

Oracle

Perfil oracle não executa SQL automaticamente. Use schema autorizado com as seis tabelas, ou execute oracle-create.sql uma vez em schema vazio. Não rodar recebido.sql em dados reais nem assumir que limpeza será feita. DDL Oracle pode efetivar commits implícitos e deixar criação parcial se houver erro.

```
export SPRING_PROFILES_ACTIVE=oracle
export DB_URL='jdbc:oracle:thin:@//HOST:1521/SERVICE'
export DB_USER='SEU_USUARIO'
export DB_PASSWORD='SUA_SENHA'
java -jar target/TOTVS-AdvisorMind-3.0.0.jar
```

PowerShell: $env:SPRING_PROFILES_ACTIVE='oracle', $env:DB_URL='...', $env:DB_USER='...' e $env:DB_PASSWORD='...'. .env.example é referência, não carregado automaticamente. Nunca versionar credenciais. Driver mantido: ojdbc17 versão23.26.3.0.0.

OpenAI

OPENAI_API_KEY no ambiente e OPENAI_MODEL opcional (padrão herdado gpt-4o-mini). Não houve chamada paga neste trabalho. O backend usa Chat Completions, exige modelo/usage.total_tokens válidos e não salva saídas acima do limite. O plano ChatGPT não substitui a configuração da API. Reinicie ao alterar o ambiente.

14. Testes executados e limites da validação

Classe	Total	Passou	Ignorado
OpenAIClientTest	8	8	0
OracleTransactionalProbeTest	1	0	1
SqlScriptTest	1	1	0
DomainRulesTest	3	3	0
RelationalApiIntegrationTest	21	21	0
OracleReadOnlyTest	1	0	1
NoKeyApiTest	1	1	0
TOTAL	36	34	2

Zero falhas e zero erros na execução final. Os dois ignorados são exclusivamente os probes Oracle opt-in. Maven clean verify retornou0 e BUILD SUCCESS. Ambiente: Linux, Zulu21.0.8, Maven3.9.11. Dependências estavam no cache, portanto a execução usou -o; primeiro uso deve omitir essa opção.

Verificações importantes

CRUD dos recursos, identidades, relações/UK/FKs, exclusão de pais com dependentes, datas/offset/fração inválidos, limites de texto e bytes, transcrição4000/4001, CLOB com mais de4000 caracteres, mudança de repositórios, contadores, JSON derivado, rollback após INSERT, falhaSQL sem vazamento, regras legadas e simulação de comissão.

IA: três tipos com sucesso controlado, metadados válidos, timeout, erros externos, chave ausente, JSON inválido, tokens ausentes/negativos/fracionários, resultado250/251 e Unicode excessivo, prévia longa não salva, duplicidade sem nova chamada e bloqueio de alteração de conteúdo já analisado.

JAR e reinicialização

52 verificações HTTP reais, três inicializações e conferência das seis tabelas persistidas. O provedor de IA era um servidor HTTP local controlado com fixtures claramente identificadas. Quatro chamadas controladas: três persistências e uma prévia. Banco/arquivos temporários do smoke não afetam dados do usuário.

Ocorrência corrigida

Um teste inicialmente falhou: o parser aceitava Z e descartava o fuso de LocalDateTime. DTOs passaram a exigir formato estrito, e a bateria foi reexecutada com sucesso. Logs de erroSQL durante testes de falha são esperados. Comandos, resultados e limites completos estão em RELATORIO-TESTES.md.

15. Oracle, Front-End e checklist acadêmico

O que não foi comprovado

Não há conexão Oracle autorizada configurada. OracleReadOnlyTest verifica colunas por SELECT; OracleTransactionalProbeTest verifica JDBC das seis tabelas, IDs, DATE, CLOB, unicidade, update/delete e rollback em dados com prefixoTEST_. Ambos são opt-in e NÃO foram executados. O probe não cria tabela nem limpa dados; sempre reverte a transação, mas pode consumir números de identidade. Não representa teste completo da API no Oracle.

```
RUN_ORACLE_READONLY=true mvn -Dtest=OracleReadOnlyTest test
RUN_ORACLE_TRANSACTIONAL=true mvn -Dtest=OracleTransactionalProbeTest test
```

Requerem DB_URL/DB_USER/DB_PASSWORD para schema de teste autorizado. Não reconfigure a bateria H2 para um banco real: ela contém limpeza/DDL exclusivos de schemas descartáveis. Privilegios, driver, charset, NLS, collation, bloqueios e comportamento remoto ainda precisam ser conferidos.

Prints reais de telas

PENDENTE: Front-End não enviado. Não foram inventadas capturas nem reaproveitados protótipos antigos como telas atuais. A equipe precisa adicionar prints reais de cadastro/listagem/edição, reunião e análise integrados. A página estática publicada anteriormente mostra a execução da versão anterior e não foi usada como evidência desta versão.

Checklist Java Sprint4

Critério	Pontos	Estado
PDF: capa, sumário, objetivo, funcionalidades, endpoints e diagramas	Parte dos20	Preparado; nome oficial da equipe ainda precisa ser confirmado.
Prints das telas implementadas	Mesmo bloco20	Pendente: Front-End não fornecido.
Model conforme banco	10	Seis tabelas representadas; validação local aprovada; Oracle remoto pendente.
DAO e Service com validações/CRUD	20	Implementados; integridade/transações testadas localmente.
API REST necessária ao Front-End	30	35 padrões de rota documentados/testados no backend; integração visual não verificada.
Boas práticas	20	Separação, PreparedStatement, recursos/transações Spring, validações, exceções e segredos externos. Padrões específicos de sala não enviados.
ZIP e PDF via Teams	Sem item separado	Arquivos finais gerados; revisão e envio ficam com a equipe.

Não há integração com modelos Data Science alegada. O DQL recebido foi preservado/testado em H2 adaptado; itens da disciplina Database continuam dependendo da validação Oracle e das exigências de apresentação/Data Modeler da equipe.

16. Pacote final e rastreabilidade

TOTVS-AdvisorMind-Sprint4-Java-FINAL.zip contém a pasta do projeto com pom.xml, fontes, testes, perfis, SQL recebido/auxiliares, coleção Postman, requests.http, diagramas editáveis/exportados, evidências e documentação. Não contém target, banco de teste, credenciais reais ou arquivos transitórios. JAR é gerado por mvn package.

Arquivo / pasta	Conteúdo
README.md / LEIA-ME.md	Execução, configuração, limites e testesOracle opt-in.
docs/API.md / endpoints.json	Contrato completo de entrada/saída, status, validações e regras.
docs/DECISOES.md	Estratégia relacional, contadores, JSON derivado e preservação de funcionalidades.
docs/REVISAO-SQL.md	Problemas do roteiro recebido e propostas separadas.
docs/RELATORIO-ALTERACOES.md	Inventário de arquivos criados/modificados/removidos, com motivos.
docs/RELATORIO-TESTES.md	Comandos, resultados reais e limitações.
docs/evidencias	Maven, resumosSurefire, ambiente e52 chamadasHTTP.
docs/diagramas	SVG/PNG exportados e fontesMermaid editáveis.
sql	Recebido formatado; DDL separado, DQL e propostas auxiliares.
src/test e scripts/smoke_http.py	Testes reproduzíveis, probesOracle e execução real doJAR.

Referências técnicas primárias

Oracle JDBC Standards Support (19c): https://docs.oracle.com/en/database/oracle/oracle-database/19/jjdbc/JDBC-standards-support.html - solicitar colunas geradas por nome para obter identidade, em vez de ROWID.

H2 Features: https://h2database.com/html/features.html - em MODE=Oracle, DATE é tratado como TIMESTAMP(0). Modos de compatibilidade não comprovam Oracle real.

OpenAI Chat Completions API: https://developers.openai.com/api/reference/resources/chat/subresources/completions/methods/create - content/model/usage e finish_reason. Regulamento acadêmico e SQL fornecidos pelo usuário são as fontes dos requisitos, não recomendações genéricas.

Situações distintas: implementação relacional concluída; teste local aprovado; Oracle remoto não validado; Front-End não validado. Nenhuma dessas condições foi usada como prova automática das demais.